ARCHITECTURAL REFERENCE

Comprehensive Guide to Agent Memory Systems

Foundations, Multi-Layer Taxonomy, Vector Indexing, and LangGraph BaseStore Implementation

1. Executive Summary & Memory Taxonomy

In autonomous agentic AI architectures, memory determines how an agent preserves state across reasoning steps, retains factual knowledge, learns from past experiences, and enforces behavioral rules. Memory systems bridge the gap between stateless LLM inferences and stateful, self-improving multi-step systems.

Memory Type	Scope & Lifespan	Core Purpose	Infrastructure / Store
1. Working / Short-Term	Single execution / thread session	Maintains active conversation turns, intermediate tool calls, reasoning traces, and node states.	LangGraph State, `MemorySaver`, Redis / Postgres Checkpointers.
2. Episodic	Cross-session (Past events)	Recalls historical execution traces, past problems solved, and case-based resolutions.	Vector Stores, LangGraph `BaseStore` with embedding indices.
3. Semantic / Long-Term	Cross-session (Permanent facts)	Persists user profiles, technical preferences, system facts, and entity relationship graphs.	Key-Value Stores, Relational DBs, Document Stores (`PostgresStore`).
4. Procedural	Permanent (Rules & Skills)	Dictates workflow rules, SOP constraints, tool JSON schemas, and structured prompt heuristics.	System Prompts, Graph DAG logic, Schema validators, Few-shot exemplars.

2. Architectural Anatomy of the 4 Memory Tiers

1. Working Memory (In-Context State)

Tracks active execution. Governed by context limits and state reducers. Checkpointed at super-step boundaries to support state inspection, human-in-the-loop rollbacks, and recovery loops.

2. Episodic Memory (Experience Trajectories)

Stores past (Problem → Action → Observation → Solution) trajectories. Enables the agent to perform similarity searches over previous incidents and reuse verified resolutions.

3. Semantic Memory (Knowledge & Profiles)

Stores consolidated world knowledge and structured user profiles (e.g. cloud provider preferences, coding conventions, domain restrictions) across independent threads.

4. Procedural Memory (Behavior & Heuristics)

Hardcoded graph topology, JSON schema validation contracts, tool execution constraints, and behavioral policies embedded within system instructions.

3. Data Flow Between Memory Subsystems

The data flow follows a cohesive cyclical structure where **Working Memory** acts as the runtime execution kernel:

- **Ingress & Pre-fetch:** When a user invokes the agent, Procedural guidelines format the system prompt. Working memory initiates a semantic key-value lookup and vector search over Semantic and Episodic memory namespaces.
- **Reasoning & Tool Execution:** The LLM processes the combined prompt. If data is missing or actions are required, it dispatches tool calls. Tool execution results and error feedback are folded back into Working Memory as observations.
- **Memory Consolidation:** As tasks resolve, dedicated consolidation steps or tools write new facts into Semantic Memory and execution episodes into Episodic Memory via `BaseStore.put()`.

> **Checkpointer vs. BaseStore Distinction in LangGraph**
>
> **Checkpointer (`MemorySaver` / `PostgresSaver`):** Thread-isolated runtime state persistence. Serializes the full message history per `thread_id`.
> **BaseStore (`InMemoryStore` / `PostgresStore`):** Global, cross-thread hierarchical storage organized by `(namespace, key)` with vector embedding index support for semantic retrieval.

4. Vector-Indexed Semantic Memory Search

LangGraph's `BaseStore` allows embedding specific fields upon insertion and performing cosine similarity searches via `store.search()`.

```
from langchain_openai import OpenAIEmbeddings
from langgraph.store.memory import InMemoryStore

# Configure store with embedding index
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
long_term_store = InMemoryStore(
    index={
        "embeddings": embedding_model,
        "dims": 1536,
        "fields": ["content"], # Automatically embeds the 'content' field
    }
)

# Search across user's episodic or knowledge namespaces
results = long_term_store.search(
    namespace=("users", "user_101", "knowledge"),
    query="database configuration and replication",
    limit=3
)
for item in results:
    print(f"Match score: {item.score:.2f} | Content: {item.value['content']}")
```

5. End-to-End Implementation: All 4 Memory Types in LangGraph

The complete Python implementation below integrates all 4 memory models, on-demand tool calling using `InjectedStore`, and persistent checkpointing.

```
import uuid
from typing import Annotated, TypedDict
from langchain_core.messages import BaseMessage, HumanMessage, SystemMessage
from langchain_core.runnables import RunnableConfig
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import InjectedStore, ToolNode, tools_condition
from langgraph.store.base import BaseStore
from langgraph.store.memory import InMemoryStore

# --- 4. PROCEDURAL MEMORY (Behavioral Rules & Policies) ---
PROCEDURAL_GUIDELINES = (
    "You are an enterprise AI assistant. Adhere strictly to these procedural rules:\n"
    "1. Always format responses using technical, concise bullet points.\n"
    "2. Consult episodic memory when addressing system crashes or infrastructure issues.\n"
    "3. Adhere strictly to user technical preferences stored in semantic memory."
)

# --- 2. EPISODIC & 3. SEMANTIC MEMORY TOOLS ---
@tool
def record_resolved_episode(
    problem: str,
    solution: str,
    *,
    config: RunnableConfig,
    store: Annotated[BaseStore, InjectedStore()],
) -> str:
    """Saves a resolved technical incident/task into episodic memory for future recall."""
    user_id = config.get("configurable", {}).get("user_id", "default_user")
    namespace = ("users", user_id, "episodic_memory")
    entry = f"Problem: {problem} | Solution: {solution}"
    store.put(namespace, key=str(uuid.uuid4()), value={"content": entry})
    return f"Successfully saved episodic memory: {problem}"

@tool
def update_user_profile_fact(
    preference_key: str,
    value: str,
    *,
    config: RunnableConfig,
    store: Annotated[BaseStore, InjectedStore()],
) -> str:
    """Updates semantic profile memory (e.g. language preferences, tech stack)."""
    user_id = config.get("configurable", {}).get("user_id", "default_user")
    namespace = ("users", user_id, "semantic_profile")
    current_item = store.get(namespace, "profile")
    profile = current_item.value if current_item else {}
    profile[preference_key] = value
    store.put(namespace, key="profile", value=profile)
    return f"Updated user preference: {preference_key} = {value}"

tools = [record_resolved_episode, update_user_profile_fact]

# --- 1. WORKING / SHORT-TERM MEMORY (State Definition) ---
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

# Vector-indexed store for Episodic and Semantic lookups
long_term_store = InMemoryStore(
    index={
        "embeddings": OpenAIEmbeddings(model="text-embedding-3-small"),
        "dims": 1536,
        "fields": ["content"],
    }
)

# Seed Long-Term Memories
```

```
long_term_store.put(
    namespace=("users", "engineer_01", "semantic_profile"),
    key="profile",
    value={"preferred_cloud": "AWS", "primary_stack": "Python/FastAPI", "role": "Principal Engineer"}
)
long_term_store.put(
    namespace=("users", "engineer_01", "episodic_memory"),
    key=str(uuid.uuid4()),
    value={"content": "Problem: ECS task OOM during batch run | Solution: Increased Fargate memory to 4GB and
reduced chunk size to 500 records."}
)

# --- AGENT REASONING NODE ---
model = ChatOpenAI(model="gpt-4o", temperature=0).bind_tools(tools)

def agent_node(state: AgentState, config: dict, *, store: BaseStore) -> dict:
    user_id = config["configurable"].get("user_id", "default_user")
    latest_query = state["messages"][-1].content

    # Retrieve Semantic Memory (Profile)
    profile_item = store.get(("users", user_id, "semantic_profile"), "profile")
    user_profile = profile_item.value if profile_item else {}

    # Retrieve Episodic Memory (Semantic Vector Search)
    past_episodes = store.search(
        namespace=("users", user_id, "episodic_memory"),
        query=latest_query,
        limit=2
    )
    episodes_str = "\n".join([f"- {ep.value.get('content')}" for ep in past_episodes]) or "None found."

    # Construct context-enriched working prompt
    full_prompt = (
        f"{PROCEDURAL_GUIDELINES}\n\n"
        f"--- USER PROFILE (Semantic Memory) ---\n{user_profile}\n\n"
        f"--- RELEVANT EPISODES (Episodic Memory) ---\n{episodes_str}\n"
    )

    response = model.invoke([SystemMessage(content=full_prompt)] + state["messages"])
    return {"messages": [response]}

# --- GRAPH COMPILATION ---
builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)
builder.add_node("tools", ToolNode(tools))

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", tools_condition)
builder.add_edge("tools", "agent")

short_term_checkpointer = MemorySaver()
app = builder.compile(checkpointer=short_term_checkpointer, store=long_term_store)
```

6. Production Hardening & Best Practices

- **Namespacing Strategy:** Maintain distinct namespaces for distinct data scopes (e.g., `("users", user_id, "profile")`, `("teams", org_id, "policies")`).
- **Memory Eviction & Compaction:** Avoid unbounded growth in episodic stores by pruning low-utility episodes or aggregating multiple runs into generalized SOP documents.
- **Asynchronous Store Operations:** For high-throughput production workloads, utilize `AsyncPostgresStore` with connection pooling to prevent I/O blocking during memory searches.